GitHub Repo Org — Research

Consolidated outputs · 09/04/26

Daniel Rosehill

 Document: Daniel Rosehill · License: MIT

Repo Org and Portability

Repo Organisation & Portability — Initial Ideas

Date: 2026-04-09 Prompt: `prompts/run/initial/2026-04-09-repo-org-and-portability.md`

Key Findings

Four approaches, roughly ordered from lightest to most structured.

1. Symlink farm (fixes the bookmark problem)

Keep repos physically grouped in topical subfolders, but maintain a flat `~/repos/github/_flat/` directory of symlinks pointing to them. Bookmark the flat view in Dolphin — links never break when you reorganise, because the symlinks are regenerated from a manifest. A tiny script rebuilds `_flat/` from the current tree.

2. Manifest-driven layout (version-controllable)

A single YAML/JSON file is the source of truth:

```yaml
- repo: git@github.com:me/foo.git
  group: claude-code
- repo: git@github.com:me/bar.git
  group: israeli-prep
```

One script reads it and:

- clones missing repos into the right subfolder
- moves existing ones if `group:` changed
- regenerates the `_flat/` symlink farm
- regenerates Dolphin "Places" bookmarks (they live in `~/.local/share/user-places.xbel` — editable XML)

Commit the manifest to a small repo (the existing `indexing-repos/` area fits naturally). On a new machine: clone the manifest repo, run the script, done. Since the manifest lists *all* repos you care about (not just currently cloned), it doubles as a "replicate on another machine" tool — the script just skips or clones as needed.

3. Dolphin bookmarks specifically

Bookmarks break because they're absolute paths. Two fixes:

- Point bookmarks at stable `_flat/` symlinks (approach 1).
- Or have the reorg script rewrite `~/.local/share/user-places.xbel` in place when it moves a repo — it's just XML with `<bookmark href="file:///...">` entries.

4. Off-the-shelf tools

- **`ghq`** (https://github.com/x-motemen/ghq) — closest off-the-shelf fit: `ghq get` clones into a predictable path, `ghq list` enumerates, scriptable. But it enforces its own `host/user/repo` layout, which fights topical grouping.
- **`myrepos` (`mr`)** — good for bulk `git pull` across many repos, doesn't solve layout.
- A ~100-line Python script around a manifest beats these for this specific use case.

Recommendation

Combine approaches 2 + 1:

- **Manifest in a git repo** = portable structure, single source of truth.
- **Symlink farm** = bookmarks that survive reorganisation.
- Claude handles the grouping (the existing `organise-github-repos` skill), and the manifest-sync script runs afterwards to record the new layout and rebuild symlinks.

Open Questions

- Should the manifest live inside `indexing-repos/` or as its own small repo?
- Do we want per-machine overrides (e.g. some repos only cloned on the workstation, not the laptop)?
- Should GitKraken's default clone path be redirected into a `_inbox/` folder so new clones land somewhere obvious before being filed?

Sources

Reasoning from first principles plus prior knowledge of `ghq`, `myrepos`, and the KDE `user-places.xbel` format. No web sources consulted for this initial pass.

 Document: Daniel Rosehill · License: MIT

Fleet Manager Tools

Existing GitHub Repo Fleet Manager Tools

Date: 2026-04-09 Prompt: `prompts/run/subsequent/2026-04-09-fleet-manager-tools.md`

Verdict

No single tool hits all 7 requirements. Closest genuine fit is **ogit** + a bulk-ops helper (**gh-gr** or **reporemover.xyz**), plus **repo-hub/repolice** for dirty-state scanning. There is a real, ~500 LOC gap in the ecosystem here.

Genuine fits (closest to spec)

ogit — TUI+CLI, multi-provider (GitHub/GitLab), organises clones under a watched dir, bulk clone by org/filter, browser launcher actions. Hits: 1 (partial — discovers remotes, organises local), 2 (clone), 6 (partial — browser/URLs). Gap: no bulk delete, no stale/dirty/orphan views, no favourites/MRU, no fuzzy over topics.

gh-gr — `gh` extension for managing many repos at once. Hits: 7 (bulk pull/push), 3 (partial — status across all repos), 2 (partial). Gap: no TUI selection, no delete, no fuzzy/MRU/launcher.

Partial fits worth knowing

- **gh-repo-man** — fzf-powered interactive browse+clone with language/archived/fork filters, concurrent cloning, tmux/editor launch. Remote-only; no local reconciliation, no delete, no state views.
- **gh-reponark** — Bubble Tea TUI that pulls config for every repo, filters (missing CODEOWNERS, protection off, etc.). Audit axis, not lifecycle.
- **repo-hub / repolice** — Rust TUI scanning a dir tree for staged/unstaged status across all clones. **Best tool found for the "dirty clones" view.** Local-only, no remote reconciliation, no ops.
- **Repo Remover** — Web app using a token to list/sort/filter and **bulk delete or archive remote repos**. The only non-TUI tool that nails the destructive remote side with multi-select. No local awareness.
- **github-tui** — general GitHub TUI (issues/PRs/repo browse). Wrong axis.

Ruled out after checking

gh-repo-fzf, gh-star, gh-repo-list, gh-get-repos, rm3l/gh-org-repo-sync, ruralocity/gh-clone-team-repos (single-axis: clone-all or fuzzy-pick, no lifecycle), gh-dep (PR TUI), gh-repo-config / gh-repo-collab / twelvelabs/gh-repo-config / myzkey/gh-repo-settings (settings/YAML, not lifecycle), mislav/gh-delete-repo (deprecated, single-repo).

Honest conclusion

Two-tool combos worth actually trying:

1. **ogit** (inventory + clone + launcher) + **gh-gr** (bulk pull/status) + **reporemover.xyz** for occasional remote-delete sweeps.
2. Write a thin Bubble Tea / Textual wrapper around `gh api user/repos` + a watched dir walk + `git status --porcelain`. Closest forkable prior art: **ogit** (Go, provider inventory + TUI) or **repo-hub** (Rust, local dirty scanning).

Given the 2025–2026 Charm/Textual surge, this is a plausible weekend OSS project — and a tool Daniel is clearly not the only person who wants.

Sources

- wmalik/ogit
- 2KAbhishek/gh-repo-man
- admcpr/gh-reponark
- daffyzk/repo-hub
- sarumaj/gh-gr
- Repo Remover

- skanehira/github-tui
- rothgar/awesome-tuis

Multi-Machine Sync

Multi-Machine Clone-Set Sync

Date: 2026-04-09 Prompt: `prompts/run/subsequent/2026-04-09-multi-machine-sync.md` Builds on: `outputs/individual/2026-04-09-fleet-manager-tools.md`

Key Findings

This problem **does** have purpose-built tools, unlike the launcher/fleet-manager question. The category is usually called "declarative multi-repo management" or "workspace manifest tools." The trick is picking one whose safety model matches your data-loss requirement.

Genuine fits

git-repo-manager (GRM) — top pick

Hakoerber's tool. Explicitly designed for this workflow.

- **Manifest-driven:** a TOML/YAML file lists every repo, its remote, and its worktree path.
- **`grm repos sync`** reconciles the local tree to the manifest: clones missing, reports drift.
- **Safety by default:** refuses to touch worktrees with uncommitted changes or unpushed commits unless forced. This is exactly the data-loss protection you asked for.
- **GitHub/GitLab discovery:** `grm repos find github <user>` generates the manifest automatically from your GitHub account — you don't hand-maintain it.
- **Committable:** the manifest is a plain file meant to live in a git repo.
- **Worktree support** if you want fancy multi-branch checkouts per repo.

Workflow on laptop: `git pull` the manifest repo → `grm repos sync config.toml` → done. New machine: same.

ghorg

Mass-clones and keeps-in-sync every repo in a GitHub user/org. Not manifest-driven — it takes the live list from GitHub as the source of truth.

- **`ghorg clone <user> --preserve-dir`** on the laptop pulls down everything you own, updates existing clones.
- **`--prune`** removes local clones no longer on remote (opt-in, with `--prune-no-confirm` for automation — leave that off).
- Does **not** have GRM's uncommitted-change safety net by default; relies on git refusing fast-forwards when there's conflict. Read the docs carefully before using `--prune`.
- Simpler mental model than GRM: "make local match GitHub" vs GRM's "make local match manifest."

mr (myrepos)

Venerable. `.mrconfig` lists repos; `mr update` runs `git pull` across all of them; `mr fetch`, `mr status`, etc. Pre-existing clones only — no GitHub discovery. Pair with `gh repo list` to generate the config.

Partial fits

- **mani** — YAML of repos, bulk commands, tag-scoped execution. Good for "run `git pull` across all repos with tag `work`" but not a sync tool in the reconcile sense.
- **gita** — multi-repo status/exec. `gita ll` is a lovely colour-coded status view. Not reconcile-driven.
- **gh-gr** — gh extension for bulk pull/push across many repos. Great companion, not a replacement.

Honest recommendation

Use GRM as the reconcile engine. It is the only tool of the bunch whose safety model explicitly protects uncommitted changes and unpushed commits, which is your hard requirement.

Setup

1. On the desktop, one-time: `grm repos find config github --token $(gh auth token) --owner danielrosehill > ~/repos-manifest/config.toml`
2. Create `~/repos-manifest/` as its own git repo. Commit `config.toml`. Push to a private GitHub repo (e.g. `daniel-repos-manifest`).
3. **Weekly cron on the desktop** that regenerates `config.toml` from GitHub (`grm repos find config ...`), diffs, commits if changed, pushes. This keeps the tree up to date automatically as you create/archive/rename repos.
4. On the laptop: clone the manifest repo once; add a shell alias `repos-sync = 'cd ~/repos-manifest && git pull && grm repos sync config.toml'`. Run it whenever you sit down at the laptop.

Bulk pull / push

- **Bulk pull:** `grm repos sync` already fetches and updates clean worktrees; leaves dirty ones alone.
- **Bulk push:** GRM doesn't push. Pair with `gh-gr push` or `gita super push` for the rare case you actually want to push across many repos at once. (Realistically you'll almost never want this — it's a footgun. Push per-repo from the launcher.)

Data-loss safety checklist

- GRM refuses to touch dirty worktrees → safe by default.
- **Never use `ghorg --prune` or any "force" flag in your cron.** Prune only interactively.
- Before sync on the laptop, run `gita ll` (or `grm repos status`) to see what's dirty, so you can deal with it before reconcile touches anything. Add this to the alias.
- Keep a `grm repos status --json > /tmp/pre-sync.json` snapshot before every sync on laptop; diff after. Cheap audit trail.

Upgraded alias

```
repos-sync() {
  cd ~/repos-manifest || return
  git pull --ff-only || { echo "manifest pull failed"; return 1; }
  grm repos status config.toml   # show dirty repos first
  read -p "Proceed with sync? [y/N] " ans
  [[ "$ans" == "y" ]] || return
  grm repos sync config.toml
}
```

How this composes with earlier outputs

- The **SQLite launcher index** (from the search-first launcher output) and the **GRM manifest** are two views of the same fact — "repos I care about." Generate both from the same `gh repo list` call.
- GRM owns the *clone-set* layer. The launcher owns the *find-and-open* layer. The GitHub Projects integration owns the *grouping* layer. All three key off remote URL, all three survive filesystem reorganisation.
- The manifest repo is also the natural home for the launcher's exported favourites and MRU, so those travel between machines too.

Open Questions

- Should the desktop's weekly manifest-regen cron also auto-archive repos that haven't been touched in N months? Probably no — too much policy in automation, too easy to lose context.
- Should GRM's output dir structure mirror the topical grouping (`claude-code/`, `clients/`, ...)? Given earlier conclusions (filesystem layout should be irrelevant), probably use a flat `~/repos/github/<name>` and let the launcher handle grouping.
- Is there a way to exclude huge repos (models, datasets) from laptop sync to save disk? GRM supports per-repo `clone: false` or tags — investigate.

Sources

- git-repo-manager (GRM) · GRM docs
- ghorg
- myrepos (mr)
- mani
- gita
- gh-gr

DB-Backed Repo Manager

DB-Backed Repo Manager — Research Report

Date: 2026-04-09 Prompt: `prompts/run/subsequent/2026-04-09-db-backed-repo-manager.md`
Builds on: `outputs/individual/2026-04-09-repo-org-and-portability.md`

Key Findings

Nothing off-the-shelf covers all six requirements (DB-backed, auto-detect on birth, survives moves, tags/notes, semantic search, launcher actions). You'll be gluing — but the glue is small, and the stack in 2026 is quite pleasant.

1. Existing tools (vs the 6 requirements)

Tool	What it does	Local DB	Survives moves	Semantic search	Gap
ghq + fzf	Clones into canonical `host/owner/name` path, lists them	No — pure filesystem	No (path IS the key)	No	Fails auto-detect & move-survival outright
gita	Multi-repo status/exec, groups in a text file	Flat file	Paths editable	No	No auto-discover, no semantic
mani	YAML-driven repo runner with tags/projects	YAML	Yes (paths in YAML)	No	No auto-detect, no DB, no launcher actions
grm / gitrepos	Declarative config	Config file	Yes	No	Same gaps as mani
projectable	TUI project manager	No	N/A	No	Per-project only
Sourcegraph self-hosted	Full code index + semantic	Postgres + Zoekt	Yes	Yes (code-level)	Heavy Docker stack; overkill for repo-list UX; free tier shrank
JetBrains / VS Code "Project Manager" ext	IDE-scoped project list	JSON	Yes	No	IDE-locked, no auto-watch
KRunner recent docs / Kickoff	System launcher	—	—	No	Not repo-aware
dokosa	Indexes local git repos with vector embeddings	SQLite-ish + vectors	Yes	**Yes**	Search-only — no tags, no launcher, no watcher
semantic-code-search	NL search *inside* a repo	Local index	—	Yes	Per-repo, not a fleet manager

Closest partials: `mani` (tags/actions, no DB/watch/semantic) and `dokosa` (semantic over repos, no tags/launcher). Neither is a drop-in.

2. Build-your-own stack recommendation

Pragmatic, ~weekend-sized:

- **Index store:** SQLite + sqlite-vec. One file, trivial backup, composes with normal SQL for tag filters. Export `tags.yaml` + `notes/<id>.md` to a git repo for version control of the *organisation layer*.

- **Stable identity:** hash of the `origin` remote URL (fallback: first-commit SHA). **This is what makes moves safe** — the primary key is never the filesystem path.
- **Embeddings:** `sentence-transformers` (`all-MiniLM-L6-v2` or `bge-small-en-v1.5`), or `fastembed` for ONNX if you want to avoid torch. Index: repo name + description + topics + first 2–4 KB of README.
- **Watcher:** Python `watchdog` (inotify backend). Watch `~/repos/` recursively; when a new `.git/` dir appears, insert a row and enqueue an embedding job. Run a periodic reconcile that matches rows to paths **by remote URL**, so manual moves are auto-healed.
- **UI:** Textual TUI ships fastest; PySide6 if you want native KDE feel later.
- **Launcher actions:** plain `subprocess.Popen`:
  - Terminal: `konsole --workdir <path>`
  - Claude: `konsole -e bash -c "cd <path> && claude"`
  - Dolphin: `dolphin <path>`
  - VS Code: `code <path>`
  - GitKraken: `gitkraken --path <path>`

3. Semantic search specifics (2026)

Pick sqlite-vec. Reasons for a single-user desktop tool indexing a few thousand READMEs:

- One file alongside the metadata DB — no daemon, no second store.
- `vec0` virtual tables do brute-force + filtered ANN, more than fast enough under ~10k vectors.
- Composes with normal SQL: `WHERE tags LIKE '%claude%' ORDER BY distance` in one query.

When *not* to pick it:

- **LanceDB** is the right answer if you grow into code-chunk-level search (millions of vectors) or want columnar analytics. Lance + DuckDB is the 2026 power-user combo — unnecessary here.
- **Chroma** is fine but heavier and the ecosystem has cooled relative to sqlite-vec / LanceDB.
- **DuckDB VSS** works but mixes two embedded DBs for no gain.

4. KRunner integration on Plasma 6

A KRunner runner is still the right "type anywhere to jump to repo" path on Plasma 6. Options:

- **Native C++ runner** using the KRunner framework — most robust; community template exists.
- **Python D-Bus runner** — implement the `org.kde.krunner1` D-Bus interface in Python, ship a `.service` + `.desktop`. This is how most third-party Plasma 6 Python runners work now. `krunner-bridge` still works but is a Plasma 5–era shim; writing a small D-Bus runner directly is cleaner.
- **Cheapest MVP:** skip KRunner initially. Bind a global KWin shortcut to open the Textual TUI. Upgrade to a real runner once the tool proves itself.

Uncertain: the maintenance state of Plasma 6 Python D-Bus runner templates — verify before committing.

Recommendation

1. **MVP (weekend):** Python + `watchdog` + SQLite + sqlite-vec + fastembed + Textual TUI. Identity = hash of `origin` URL. Global shortcut launches the TUI.
2. **v2:** Export tags/notes to a git repo so the organisation layer is portable across machines — this subsumes the "manifest" idea from the first output.
3. **v3:** KRunner runner for system-wide invocation.
4. Keep GitKraken's default clone dir as `~/repos/github/_inbox/` so new repos land in an obvious place the watcher can pick up, then get filed (or not — filing becomes cosmetic once the DB is source of truth).

Open Questions

- Should the org-layer export repo live inside existing `indexing-repos/`?
- Do we want per-machine overrides (some repos only cloned on workstation, not laptop)?
- Is there value in also indexing *uncloned* GitHub repos (via `gh repo list`) so semantic search covers the full set, and the launcher offers "clone into inbox" as an action?

Sources

- ghq
- gita
- mani
- projectable
- dokosa — semantic search over local git repos
- sturdy-dev/semantic-code-search
- sqlite-vec
- sqlite-vec overview
- LanceDB
- Lance × DuckDB, Jan 2026
- Embedded DBs comparison
- KRunner dev docs (Plasma 6)
- Plasma 6 KRunner template thread
- krunner-bridge
- watchdog
- fastembed
- Textual

 Document: Daniel Rosehill · License: MIT

GUI Runner for Repos

GUI Runner for Repos

Date: 2026-04-09 Prompt: `prompts/run/subsequent/2026-04-09-gui-runner-for-repos.md`

Verdict

No turnkey tool exists that hits all four requirements (base-path scoping + fuzzy + frecency bias + per-result action hotkeys) as a single install. But the gap is small — ~50 lines of script hosted inside an existing GUI runner. Three viable hosts, ordered by shortest-path-to-working.

Closest turnkey partials

- **sesh** — tmux session manager, fuzzy-searches directories with zoxide frecency built in. If you lived in tmux this would be 90% of the answer. No GUI runner popup; no per-result action hotkeys beyond "attach session." Worth knowing about.
- **zoxide + `zi`** — CLI frecency jumper. Not a GUI runner. Complementary for terminal use, not a replacement.
- **Albert's Files extension** — can fuzzy-index a base path. No frecency bias, no per-result action hotkeys beyond Albert's default Enter/Shift+Enter/Ctrl+Enter triad.
- **Ulauncher File Search extension** — same shape, same gap.
- **KRunner built-in** — can match directory names but has no way to scope to a base path, no frecency, no custom per-result actions without writing a runner.

None are turnkey. Every one of them would need scripting on top to add frecency + the four-action hotkey set.

Host launcher — shortest path to working

Option 1: Rofi + script (15 minutes, ugly but works)

```
#!/usr/bin/env bash
# ~/.local/bin/repo-runner — bind to Meta+R in KWin
DB=~/.local/share/repo-index.db
sqlite3 "$DB" <<'SQL' > /tmp/repos.list
SELECT printf('%s\t%s\t%s', name, local_path, remote_url)
FROM repos
WHERE local_path IS NOT NULL
ORDER BY open_count DESC, last_opened DESC NULLS LAST, name;
SQL

sel=$(rofi -dmenu -i -p "repo" \
  -kb-custom-1 'Alt+t' \
  -kb-custom-2 'Alt+v' \
  -kb-custom-3 'Alt+f' \
  -kb-custom-4 'Alt+c' \
  -format 'i:s' \
  < <(cut -f1 /tmp/repos.list))

key=$?
[ -z "$sel" ] && exit 0
idx="${sel%%:*}"
line=$(sed -n "$((idx+1))p" /tmp/repos.list)
path=$(cut -f2 <<<"$line")
url=$(cut -f3 <<<"$line")

case "$key" in
  10) konsole --workdir "$path" ;;                          # Alt+t
  11) code "$path" ;;                                       # Alt+v (IDE)
  12) dolphin "$path" ;;                                    # Alt+f
  13) konsole --workdir "$path" --new-tab -e claude ;;      # Alt+c (both)
```

```
  *)  konsole --workdir "$path" -e bash -c 'claude; exec bash' ;;    # Enter
esac

sqlite3 "$DB" "UPDATE repos SET open_count = open_count + 1,
  last_opened = datetime('now') WHERE remote_url = '$url';"
```

Bind `Meta+R` → this script in System Settings → Shortcuts → Custom Shortcuts. Done. Rofi gives you:

- Base-path scoping (SQL filter).
- Fuzzy match (`-i` , and `-matching fuzzy`).
- Frecency bias (`ORDER BY open_count DESC` in the query).
- Per-result action hotkeys (`-kb-custom-N`).
- A real GUI popup.

This is the **minimum viable GUI runner** and the shortest path from "no tool" to "daily driver."

Option 2: Albert Python extension (half a day, polished)

Albert is a better long-term host because per-result actions are a first-class concept in its API — each returned `Item` carries an `actions=[...]` list, and Albert renders them as Enter, Shift+Enter, Ctrl+Enter with proper labels. The extension is ~60 lines of Python:

```
# ~/.local/share/albert/python/plugins/repos/__init__.py
from albert import *
import sqlite3, subprocess, os

md_iid = "2.3"
md_name = "Repos"
md_description = "Fuzzy-open local GitHub repos"
md_id = "repos"

class Plugin(PluginInstance, TriggerQueryHandler):
    def __init__(self):
        PluginInstance.__init__(self)
        TriggerQueryHandler.__init__(self, self.id, self.name,
            self.description, defaultTrigger="r ")

    def handleTriggerQuery(self, query):
        db = sqlite3.connect(os.path.expanduser("~/.local/share/repo-index.db"))
        rows = db.execute("""
            SELECT name, description, local_path, remote_url
            FROM repos
            WHERE local_path IS NOT NULL AND name LIKE ?
            ORDER BY open_count DESC, last_opened DESC
            LIMIT 25
        """, (f"%{query.string}%",)).fetchall()

        for name, desc, path, url in rows:
            query.add(StandardItem(
                id=url, text=name, subtext=desc or path,
                iconUrls=["xdg:folder-git"],
                actions=[
                    Action("claude", "Open in Claude Code",
                        lambda p=path, u=url: self.launch(
                            ["konsole", "--workdir", p, "-e",
                             "bash", "-c", "claude; exec bash"], u)),
                    Action("term", "Open terminal",
                        lambda p=path, u=url: self.launch(
                            ["konsole", "--workdir", p], u)),
                    Action("both", "Claude + terminal",
                        lambda p=path, u=url: self.launch(
```

```
                            ["konsole", "--workdir", p, "--new-tab",
                             "-e", "claude"], u)),
                    Action("dolphin", "Open in Dolphin",
                        lambda p=path, u=url: self.launch(
                            ["dolphin", p], u)),
                    Action("code", "Open in VS Code",
                        lambda p=path, u=url: self.launch(
                            ["code", p], u)),
                ]))

    def launch(self, cmd, url):
        subprocess.Popen(cmd)
        db = sqlite3.connect(os.path.expanduser("~/.local/share/repo-index.db"))
        db.execute("""UPDATE repos SET open_count = open_count + 1,
            last_opened = datetime('now') WHERE remote_url = ?""", (url,))
        db.commit()
```

Trigger becomes `r <query>` in Albert. The frecency ordering is in the SQL. Actions are keyboard-navigable via Albert's standard UI.

Option 3: KRunner runner (don't — yet)

For system-wide "type anywhere, no trigger word needed" it would be the most polished result. But per-result Alt-key actions are awkward in KRunner and the Plasma 6 Python runner template story is still shaky (flagged in the earlier launcher output). Revisit if Albert isn't enough.

The index behind both options

Both options read from a single `~/.local/share/repo-index.db`:

```sql
CREATE TABLE repos (
  remote_url TEXT PRIMARY KEY,   -- stable identity
  name TEXT NOT NULL,
  owner TEXT,
  description TEXT,
  topics TEXT,
  local_path TEXT,               -- NULL if uncloned
  open_count INTEGER DEFAULT 0,
  last_opened TEXT
);
```

Populated by a ~50-line Python script on a cron:

- `gh repo list danielrosehill --limit 1000 --json ...` → upsert metadata.
- `fd -t d -H '^\.git$' ~/repos/github/` + `git -C <parent> remote get-url origin` → fill `local_path` by matching on remote URL.
- This is the **same DB** as the fleet-manager launcher design (earlier output). One index, multiple front ends.

Recommended sequence

1. **Today (15 min):** build the SQLite index + Rofi script. Bind `Meta+R`. Ship it.
2. **Next weekend:** write the Albert extension reading the same DB. Keep both working so you can compare.
3. **Later:** add uncloned-repos to search results with a "Clone into `~/repos/github/_inbox/` then open" action, so the runner becomes a unified find-and-open for every repo you own, not just cloned ones.

Composes with other outputs

- Index → shared with the search-first launcher design.
- `local_path` updates → driven by the `systemd.path` watcher from the auto-symlink output.

- Remote-URL identity → matches GRM's clone-set sync model; filesystem reorganisation never breaks the runner.
- Frecency column → travels between machines if `~/.local/share/repo-index.db` is synced via Syncthing, giving you the same MRU ordering on laptop and desktop.

Open Questions

- Should Rofi also show an inline indicator for dirty repos (uncommitted changes)? Cheap: add a `status` column refreshed by the same cron.
- Is Rofi on Wayland (KDE Plasma 6) actually working well now, or is `rofi-wayland` / `wofi` needed? Verify before committing.
- Does Albert 2.x run cleanly on Plasma 6 Wayland? Worth a quick test before investing in the extension.

Sources

- Rofi custom keybindings — `-kb-custom-N`
- Albert launcher Python extensions
- sesh
- zoxide
- rofi-wayland
- KWin custom shortcuts

Search-First Launcher

Search-First Launcher — Design

Date: 2026-04-09 Prompt: `prompts/run/subsequent/2026-04-09-search-first-launcher.md` Builds on: `outputs/individual/2026-04-09-autosymlink-existing-tools.md`

Key Findings

This is the right reframe. **Stop trying to make the filesystem the index.** Let Claude reorganise folders freely — the launcher doesn't care because repo identity is `git remote get-url origin`, not a path. Bookmarks become obsolete.

The workflow collapses to three layers:

1. **An index** of every repo you care about (cloned or not), keyed by remote URL, with name + description + topics + GitHub Project membership.
2. **A launcher** with fuzzy search over the index.
3. **Action hotkeys** on each result for Claude / Konsole / VS Code / Dolphin / browser / combos.

Launcher choice

Ranked for this use case on KDE Plasma 6:

Launcher	Pro	Con	Verdict
Albert	Mature Python extension API, per-result **actions** are a first-class concept (Enter + Alt+1/2/3 etc.), cross-DE, actively maintained	Not KDE-native; one more background process	**Top pick** — actions model fits exactly
Ulauncher	Very easy Python extensions, nice UI	Actions per result are clunkier than Albert; GTK-leaning	Strong runner-up
KRunner	Native Plasma 6, type-anywhere via default shortcut	Per-result alt-key actions are awkward; Python D-Bus runner template shaky (flagged in prior output)	Use later as a secondary entry point, not primary
Rofi + script	Ultra-lightweight, scriptable; `-kb-custom-N` gives per-key actions	Bespoke, you maintain the rendering	Good minimalist option
fzf TUI bound to global shortcut	Trivial to build, `--expect` gives you per-key actions out of the box	Terminal popup feels less polished	**Best MVP** — 20 lines of shell

Recommendation: start with the fzf TUI (same-day MVP), migrate to Albert once you want a prettier always-available bar.

The index

One SQLite file, three sources merged:

1. **`gh repo list <you> --limit 1000 --json nameWithOwner,description,repositoryTopics,url,sshUrl`** — every GitHub repo you own, cloned or not.
2. **`gh project list`** + **`gh project item-list`** — membership in each Project, joined on repo URL. This gives you topical grouping **without maintaining anything locally** — you already curate Projects, so reuse that curation. Refresh on a cron (hourly is fine).
3. **Local clone locations** — `fd -t d -H '\.git$' ~/repos` once, then `systemd.path` updates. Matched to index rows by `git remote get-url origin`. If a repo is in the index but not cloned, the launcher offers "Clone into `~/repos/github/_inbox/`" as an action.

Columns: `remote_url PK, name, owner, description, topics, projects, local_path NULLABLE, last_opened, open_count`.

Sort results by `open_count DESC` so your 10 daily repos float to the top — solves the "same 10 repos 99% of the time" case without manual pinning.

Actions (hotkey bindings on a selected result)

Key	Action	Command
`Enter`	Open in Claude Code	`konsole --workdir <path> -e bash -c 'claude; exec bash'`
`Alt+T`	Terminal only	`konsole --workdir <path>`
`Alt+C`	Claude + terminal side-by-side	`konsole --workdir <path> --new-tab -e claude` then another tab
`Alt+V`	VS Code	`code <path>`
`Alt+I`	JetBrains (IDEA/PyCharm)	`idea <path>` / `pycharm <path>`
`Alt+F`	Dolphin	`dolphin <path>`
`Alt+G`	GitHub page in browser	`xdg-open <repo_url>`
`Alt+K`	GitKraken	`gitkraken --path <path>`
`Alt+N`	Clone into `_inbox/` (if uncloned)	`gh repo clone <nwo> ~/repos/github/_inbox/<name>`

Every action bumps `open_count` and `last_opened`.

Minimum viable stack (fzf version — ship today)

```
# ~/.local/bin/repo-launch
#!/usr/bin/env bash
set -e
DB=~/.local/share/repo-index.db
choice=$(sqlite3 -separator $'\t' "$DB" \
  "SELECT name, description, local_path, remote_url
   FROM repos ORDER BY open_count DESC, name" \
  | fzf --delimiter=$'\t' \
        --with-nth=1,2 \
        --expect='alt-t,alt-v,alt-f,alt-g,alt-c,alt-n' \
        --preview='echo {} | cut -f2')

key=$(head -1 <<<"$choice")
row=$(tail -1 <<<"$choice")
path=$(cut -f3 <<<"$row")
url=$(cut -f4 <<<"$row")
```

```
case "$key" in
  alt-t) konsole --workdir "$path" ;;
  alt-v) code "$path" ;;
  alt-f) dolphin "$path" ;;
  alt-g) xdg-open "$url" ;;
  alt-c) konsole --workdir "$path" --new-tab -e claude ;;
  alt-n) gh repo clone "${url#*github.com/}" ~/repos/github/_inbox/ ;;
  *)     konsole --workdir "$path" -e bash -c 'claude; exec bash' ;;
esac

sqlite3 "$DB" "UPDATE repos SET open_count = open_count + 1,
  last_opened = datetime('now') WHERE remote_url = '$url'"
```

Bind to `Meta+R` in KWin → System Settings → Shortcuts → Custom Shortcuts. Done.

Separately:

- **`repo-index-sync`** — cron every hour: `gh repo list` + `gh project item-list` → upsert SQLite. ~50 lines of Python.
- **`repo-index-watch`** — `systemd.path` on `~/repos/github/` runs a tiny script that updates `local_path` for any new `.git/` by reading its remote.

Albert v2 (when you want the nicer UX)

- Write a small Python extension reading the same SQLite DB.
- Each query returns an Albert `Item` with an `actions=[...]` list — Albert renders Enter + Shift+Enter + Ctrl+Enter as primary/secondary/tertiary natively. For more than three, use prefix queries (`r claude` → Claude, `rt claude` → terminal).
- Keeps the same DB, same sync scripts — only the front end changes.

Why this is sustainable

- **Filesystem chaos is fine.** Claude can shuffle folders daily; the reconciler re-matches by remote URL. No bookmarks to break.
- **GitHub Projects is the organisation layer.** You already curate it. No second system to maintain.
- **Frecency solves the 10/hundreds split** without manual pinning.
- **Uncloned repos are first-class.** Search finds them; a hotkey clones them. The launcher is a unified view of “all repos I might touch”, not “repos currently on disk”.
- **Every piece is replaceable.** fzf → Albert → KRunner migrations don’t require rebuilding the index.

Open Questions

- Should the SQLite DB also sync across machines (Syncthing on `~/.local/share/repo-index.db`)? Open-count telemetry would then follow you.
- Worth indexing README head text for semantic search too, or is fuzzy-on-name+description+topics enough?
- GitHub Projects (v2) has a GraphQL-only API for some fields — does `gh project item-list` expose enough, or do we need a `gh api graphql` call?

Sources

- Albert launcher · Albert Python extension docs
- Ulauncher
- fzf `--expect` key bindings
- gh repo list · gh project
- systemd.path
- KWin custom shortcuts

Autosymlink Existing Tools

Existing Tools for Auto-Symlinking New Repos

Date: 2026-04-09 Prompt: `prompts/run/subsequent/2026-04-09-autosymlink-existing-tools.md`

Key Findings

Short answer: no single existing tool does all three things. But you can compose 2–3 boring, well-maintained building blocks to get 90% there without writing an application.

The honest tension: *symlinks themselves don't survive moves* — if the target directory is moved, the symlink becomes a dangling link. Anything that "makes symlinks survive moves" is really something that **re-points the symlink after a move is detected**. So the realistic options split into two camps: (a) keep symlinks and add a reconciler, or (b) skip symlinks and use a tool that indexes by something more stable than path.

Option A — Symlink farm + watcher (closest to what you asked for)

Three boring pieces:

1. **`systemd.path` unit** or **`inotifywait`** watches `~/repos/github/` for new directories. `systemd.path` is the more modern/robust choice — it triggers a `.service` when a path changes. No daemon you wrote yourself.
2. **GNU Stow** manages the symlink farm. You keep a "package" directory per category (`~/repos/_links/claude-code/`, `_links/clients/`, etc.) and Stow creates/removes symlinks cleanly. It's battle-tested (dotfile managers have used it for 20 years) and handles add/remove without leaving orphans.
3. **A ~30-line reconcile script** runs on the `systemd.path` trigger *and* on a timer. It:
   - detects new `.git/` dirs and prompts (or auto-files by rules) which category symlink farm to add them to,
   - walks existing symlinks, detects dangling ones, searches `~/repos/` for the moved target by remote URL (`git -C <dir> remote get-url origin`), and re-points the symlink.

This gives you:

- ✅ auto-symlink on birth (systemd.path)
- ✅ categorised symlink stores (Stow packages)
- ✅ folder symlinks survive moves (reconciler re-points by remote URL)
- ✅ bookmarkable in Dolphin (bookmark the Stow target dir, never the underlying repo)

It's not an "existing tool" in the single-install sense, but every piece is off-the-shelf and the glue is tiny. This is genuinely the pragmatic minimum.

Option B — Skip symlinks entirely

If the goal is really just "find and jump to a repo fast without caring where it lives", symlinks may be the wrong primitive:

- **zoxide** — frecency-based directory jumper. Learns the dirs you actually use; `z claude` jumps to the most-used matching dir. Handles moves naturally (new location just replaces old in the index). Terminal/Claude-launch use case solved cleanly. **Does not help Dolphin bookmarks.**
- **fzf + `fd`** — `fd -t d -H '\.git$' ~/repos | fzf` gives you a live fuzzy picker over every repo. Bind to a global shortcut via KWin + a small wrapper that opens Dolphin/Konsole/Claude at the choice. No symlinks, no reconciler, no DB.
- **KDE Plasma "Places"** — you can add categorised entries in the Places panel manually. Painful to maintain but zero tooling.

Option C — Tools that almost fit but don't

- **`ghq`** — clones into a canonical `host/owner/name` path. Doesn't watch, doesn't symlink, doesn't categorise, and the canonical path fights your topical grouping. Not it.
- **`autofs`** — mounts on access. Wrong abstraction; meant for network/removable filesystems.
- **`bindfs` / bind mounts** — have the same problem as symlinks (target move breaks them) plus require root and fstab.
- **`projectile` / `mani` / `gita`** — config-file based, no auto-detect, no symlinks.
- **Nextcloud / Syncthing "favourites"** — not repo-aware.

Recommendation

Do Option A with a specific, minimal stack:

```
~/repos/github/                  ← real repos live here (flat or grouped, doesn't matter)
~/repos/_links/
  claude-code/                   ← Stow package (bookmark this in Dolphin)
  clients/                       ← Stow package
  research/                      ← Stow package
  _all/                          ← Stow package containing every repo
```

- `systemd.path` unit watches `~/repos/github/` → runs `repo-reconcile.sh`.
- `repo-reconcile.sh`:
  1. For each new `.git/` found, add to `_all/` unconditionally; prompt or rule-match for topical package.
  2. Walk every symlink under `~/repos/_links/`, test with `[ -e link ]`, and for any dangling one, find the new location by matching `git remote get-url origin` across `~/repos/`, then `stow -R` to re-point.
- Stow handles the actual symlink mechanics, so the script stays trivial.
- The `_links/` tree itself can be a git repo — that version-controls your organisation structure across machines (minus the targets, which are recreated by the reconciler on first run).

And layer **zoxide** on top for terminal/Claude jumps — it's complementary, not competing, and takes 30 seconds to install.

Open Questions

- Do you want the reconciler to be interactive on new-repo birth, or purely rule-based (e.g. "if repo name contains `claude` → `claude-code/` package")?
- Should `_links/` live in Dolphin's Places panel as a single entry, with users drilling into subcategories, or should each category be pinned separately?

Sources

- GNU Stow
- systemd.path units
- inotifywait (inotify-tools)
- zoxide
- fzf · fd
- ghq

Natural-Language Local AI Assistant for `~/repos/github/`

Date: 2026-04-09 Status: Exploratory — alternative frame to the settled Rofi+SQLite+GRM stack in `outputs/final/2026-04-09-repo-management-synthesis.md`.

Key Findings

- **Nothing off the shelf does this well as of April 2026.** Every candidate is either (a) a generic "chat with documents" wrapper (AnythingLLM, LM Studio RAG, GPT4All, FolderChat, LlamaIndex RAG CLI) that indexes file *contents* but has no concept of "repo" as a first-class entity, no frecency, and no action layer; or (b) a code-assistant (Continue.dev, aider, Cursor, Cody) that is scoped *inside* one repo, not *across* a fleet of hundreds. None of them answer "the repo where I was messing with whisper fine-tuning" and then open it in Claude Code.
- **The right move is to build a thin NL layer over the SQLite index the settled stack already produces.** The index is the hard part; it already exists (or will in a week). Adding natural-language search is a 150-line Python script: Ollama with a small tool-calling model, two tools (`search_repos`, `open_repo_action`), and the same DB. No new vector store required for the MVP — fuzzy-match on metadata with LLM-reranked top-K is sufficient and dramatically faster than embedding RAG.
- **This augments rather than replaces Rofi.** Rofi remains the 200ms muscle-memory path for "I know the repo name." The NL assistant is the escape hatch for "I don't remember the name but I remember what it was about." Two different cognitive modes, same backing DB.
- **Recommended model:** `qwen2.5:7b-instruct` or `qwen2.5:14b-instruct` via Ollama — both have robust tool-calling in 2026 and fit comfortably on a capable desktop. Avoid llama3.x for tool calling (still flaky per community reports).

1. Existing tools surveyed

Tool	What it is	Fit for this use case	Verdict
AnythingLLM	Desktop app, file-search agent, multi-workspace RAG	Can be pointed at `~/repos/github/` and granted filesystem access; agent can grep/list files. No repo-identity concept, no frecency, no GitHub Projects awareness, no per-result action hotkeys. Heavy GUI.	Overkill + wrong primitive. Close but misses the model.
LM Studio (RAG mode)	Local LLM GUI with document chat	Same issue: indexes file contents, not repo metadata. No action layer.	No.
GPT4All / PrivateGPT variants	Local RAG over a folder	Same as above. Document-centric, not entity-centric.	No.
FolderChat	Tree-checkbox RAG over folders, Gemma embeddings	Windows-first, and still document-RAG not repo-RAG.	No.
LlamaIndex `rag` CLI	Terminal RAG REPL over a directory	Closest *framework* but not a turnkey answer; you'd still build the repo-entity layer yourself. If building, LlamaIndex or pure Python+sqlite is a coin-flip — pure Python wins on leanness.	Framework option if building.
Continue.dev / aider / Cursor / Cody	In-repo code assistants	Scoped *inside* a single repo. Wrong scope entirely.	No.
Sourcegraph Cody (self-hosted)	Code search across repos with LLM	Actually does cross-repo semantic code search, but it's a heavy Docker stack, content-focused not metadata-focused, and ruled out in prior iteration as overkill.	No.
Ollama + `llm` CLI (Simon Willison) + `llm-embed-ollama`	CLI plumbing for embeddings/chat	Building blocks, not a product. Useful if building.	Building block.
txtai	All-in-one semantic search + LLM orchestration framework	Interesting if you later want semantic search over READMEs. Overkill for MVP.	Later, maybe.
KDE/Plasma native options	KRunner, Baloo	Baloo indexes file contents but has no LLM/NL layer and no repo semantics. KRun-	No.

Conclusion: no turnkey tool. The gap is that all "chat with filesystem" tools model documents, and all code assistants model *one* repo. Nothing models "a fleet of repos as searchable entities with actions." This is the same ~500 LOC gap the previous synthesis identified, just from a different angle.

2. Minimal bootstrap architecture

Reuse the SQLite index from the settled stack. Add one script. That's it.

```
┌──────────────────────────────────────────────────────────┐
│  NL CLI / hotkey     repo-ask "the whisper fine-tune one" │
│   ↓                                                      │
│  Ollama (qwen2.5:7b-instruct, tool-calling)              │
│   ↓ tools:                                               │
│      search_repos(query, k=8)     → SQL LIKE/FTS5 on DB  │
│      open_repo(remote_url, action) → shells out          │
│   ↑ reads                                                │
│  ~/.local/share/repo-index.db  (the same SQLite index)   │
└──────────────────────────────────────────────────────────┘
```

Components

1. **Index (unchanged):** the existing `~/.local/share/repo-index.db` with `name`, `description`, `topics`, `projects`, `last_commit_date`, `open_count`, `local_path`. Enable an FTS5 virtual table over `name || description || topics || projects` — trivial to add and gives sub-5ms fuzzy lookup over thousands of rows.
2. **LLM runtime:** Ollama. Model = `qwen2.5:7b-instruct` (fast, solid tool calling) or `qwen2.5:14b-instruct` if the machine has headroom. Avoid llama3.x and gemma2 for tool calling in 2026 — community reports still flag reliability issues. Alternative: `gpt-oss:20b` if Daniel already has it pulled.
3. **Two tools, defined in Ollama tool-calling JSON schema:**
   - `search_repos(query: str, k: int = 10)` — runs `SELECT ... FROM repos_fts WHERE repos_fts MATCH ? ORDER BY bm25(repos_fts) * (1 + open_count*0.01) DESC LIMIT ?`. Returns name + description + topics + local_path + remote_url. The LLM does the reranking and picks the best match from candidates; no embedding RAG needed at this scale.
   - `open_repo(remote_url: str, action: Literal["claude", "terminal", "vscode", "dolphin", "github"])` — shells out to the matching command (`konsole -e claude`, `code`, `xdg-open`, etc.). Bumps `open_count` on success. This is identical to the Rofi action layer — literally the same shell functions.
4. **The NL loop:** ~100 lines of Python. Takes a query, sends it + tool schemas to Ollama, executes tool calls, loops until the LLM emits a final answer with a concrete action. Stream tokens to the terminal so it feels live. Optional confirmation step before executing destructive or ambiguous actions.
5. **Exposure surfaces:**
   - **CLI:** `repo-ask "whisper fine-tune"` — the primary, scriptable entry point.
   - **Hotkey:** `Meta+Shift+R` bound via KDE Custom Shortcuts → runs `konsole -e repo-ask` with a small input dialog (`kdialog --inputbox`), then hands off to the CLI.
   - **Later:** Albert extension that delegates "?" prefixed queries to `repo-ask`, so Albert stays instant-fuzzy for known-name lookups but gains NL as a fallback mode.
   - **Not recommended yet:** KDE Plasma widget. Plasmoid dev on Plasma 6 is still rough; CLI+hotkey covers the value at 1/10 the effort.

Why no vector DB in the MVP

At hundreds of repos with ~100-500 chars of metadata each, total corpus is ~100KB. FTS5 full-text search over that is effectively free and returns in microseconds. Embedding RAG adds: an ingestion pipeline, stale-embedding problems, a second dependency (chroma / sqlite-vec), and does not meaningfully improve recall when the LLM itself is doing the final rerank over the top-20 FTS candidates. **Add embeddings later only if fuzzy-over-metadata stops recalling the right repo** — which the settled stack already assumed wouldn't happen for 99% of queries.

Optional v2: README-aware retrieval

If NL queries start failing on repos where the README describes the topic but the name/description doesn't (e.g. "the one where I tried ROCm with that custom kernel" and neither name nor description mention ROCm), add: - `sqlite-vec` extension + `nomic-embed-text` via Ollama - Embed the first 2KB of each README on index update - Extend `search_repos` to do hybrid FTS + vector search - Est. ~50 additional LOC, ~5 min one-time ingestion for 500 repos

Flag this as v2, not v1. The previous synthesis deliberately parked README semantic search as premature — that judgment still holds for the MVP.

3. Trade-offs vs the settled Rofi+SQLite+GRM stack

Dimension	Rofi launcher	NL assistant
Latency	~50-100ms end-to-end	~1-4s end-to-end (Ollama inference + tool call + action). Tolerable for fuzzy-intent queries; unacceptable for muscle-memory "I know the name" lookups.
Accuracy on known names	Near perfect (fuzzy match + frecency)	Perfect but wasteful — LLM for a substring match is silly.
Accuracy on fuzzy intent	Bad — you must know a keyword that's in name/description/topics. "whisper fine-tune" only hits if you tagged it.	Good — the LLM can rerank on partial recall, synonyms, and context ("that israeli prep stuff" → matches repos tagged `hebrew`, `aliyah`, `israel`).
Action execution	First-class via `-kb-custom-N`	Via LLM tool calling — works but adds one round-trip.
Cost of staleness	Index staleness hurts both equally (same DB).	Same.
Cognitive load	Hotkey + 3 keystrokes + Enter.	Hotkey + type a sentence + read + confirm.
Replaces the frecency launcher?	—	**No. Augments.** Rofi is the default, NL is the "I don't remember" fallback.

Recommendation: ship Rofi first (as already planned), then add `repo-ask` as a second entry point a week later. They share the DB and the action layer entirely — there's no duplicated work. The NL assistant is additive, scoped, and cheap.

4. How action execution actually works

Ollama's tool-calling API (stable since late 2024, well-supported in 2026) expects the client to:

1. Send the user query + a list of tool JSON schemas to `/api/chat`.

2. Receive back either a final message or one or more `tool_calls` entries with `name` + `arguments`.
3. Execute the tool client-side. For `open_repo(remote_url="git@github.com:danielrosehill/whisper-finetune", action="claude")`:

```
row = db.execute("SELECT local_path FROM repos WHERE remote_url=?",
(remote_url,)).fetchone()
subprocess.Popen(["konsole", "--workdir", row["local_path"], "-e", "claude"])
db.execute("UPDATE repos SET open_count=open_count+1, last_opened=? WHERE remote_url=?",
(now, remote_url))
```

4. Append the tool result message and send back to Ollama to get the final natural-language confirmation.

Confirmation gate: for any action, print the matched repo + proposed action and require Enter before executing. One extra keystroke; eliminates the whole class of "LLM opened the wrong thing" failures. Uncertain: whether to also gate read-only surfacing ("here's what I think you mean") — probably yes for ambiguous queries (top-1 score < threshold), no for confident ones.

Claude Code specifically: `konsole --workdir <path> -e claude` or `konsole --workdir <path> -e bash -c 'claude; exec bash'` if you want the shell to persist after Claude exits. Daniel's existing Rofi action script will already have this one-liner — reuse it verbatim.

5. Concrete build plan (if greenlit)

Assumes the Rofi+SQLite MVP from the synthesis is already in place (if not, build that first — it's a prerequisite).

1. **Day 1 (1 hour):** add FTS5 virtual table to `repo-index.db`; populate alongside existing rows in the hourly cron.
2. **Day 1 (30 min):** `ollama pull qwen2.5:7b-instruct`; smoke-test tool calling with a 20-line script.
3. **Day 2 (2 hours):** write `repo-ask` — ~150 lines of Python: argparse, Ollama client, two tool functions, loop, confirmation prompt, action dispatch reusing the Rofi action shell functions.
4. **Day 2 (15 min):** KDE Custom Shortcut `Meta+Shift+R` → `kdialog --inputbox "repo?" | xargs -I{} konsole -e repo-ask "{}"`.
5. **Week 2:** measure. How often does NL beat Rofi? Which queries fail? If README-blind failures dominate, add the hybrid embedding path from §2.

Total net-new effort over the already-planned stack: **~4 hours**. This is the cheapest feature in the whole research iteration.

6. Uncertainties and open questions

- **Tool-calling reliability of 7B-class local models on real-world queries.** Community consensus in 2026 is "qwen2.5 works, most others are flaky" but this is a moving target — benchmark on Daniel's actual queries before committing to a model. `gpt-oss:20b` may outperform if already available.
- **Whether the LLM will hallucinate remote URLs** when the FTS result is ambiguous. Mitigation: the `open_repo` tool validates `remote_url` exists in the DB and refuses unknown keys. Belt-and-braces.
- **Whether 1-4s latency is acceptable as a fallback.** Unknown until used. If it feels slow, swap to a 3B model (`qwen2.5:3b-instruct`) — may lose some reranking quality but stay under 1s.
- **Voice input layer.** Out of scope here, but a whisper.cpp front-end piping to `repo-ask` would make "open my whisper fine-tuning repo" a literal voice command. Trivial addition once the text path works. Flag as a fun v3.
- **Publishing as OSS.** The same note from the prior synthesis applies — this is ~650 LOC total including the NL layer, and the NL layer is the part most differentiating for prospective users. Worth considering.

7. Recommendation

Build the minimal NL assistant as an addition to the settled stack, not a replacement. Specifically:

1. Ship Rofi + SQLite + GRM as already planned — that's the 200ms daily driver.
2. Add `repo-ask` (Ollama + qwen2.5:7b + two tools + FTS5) as a secondary Meta+Shift+R entry point. ~4 hours of work over the planned stack.
3. Defer embedding-based README retrieval until fuzzy+FTS+LLM-rerank demonstrably fails.
4. Do not adopt AnythingLLM / LM Studio / FolderChat / any other "chat with filesystem" tool — they model the wrong primitive and would require more work to bend into shape than building the thin layer from scratch.

The key insight: **the hard part of this problem is the repo-entity index, not the NL layer.** The settled stack already solves the hard part. Adding NL on top is small, cheap, and additive — and it's the only form of "local AI assistant over my repos" that will actually feel like it understands repos rather than files.

Sources

- AnythingLLM agent usage
- LlamaIndex RAG CLI
- FolderChat
- LM Studio RAG docs
- Ollama tool calling (IBM tutorial)
- Ollama embeddings docs
- Ollama library (qwen2.5, gpt-oss)
- llm-embed-ollama plugin
- local-llm-rag-chromadb example
- txtai framework
- Ollama web search and agent capabilities (DeepWiki)
- Ask Your Codebase Anything with Ollama + RAG (Medium)
- Prior research: `outputs/final/2026-04-09-repo-management-synthesis.md` and `SCOPE.md` in this workspace.

Repo Quick-Launcher

Repo Quick-Launcher with Action Menu

Key Findings

No single off-the-shelf tool bundles "fuzzy-find a repo + present an action menu + multiplex Claude/terminal/files in a tiled window" on KDE Wayland. But every piece exists and the glue is ~40 lines of bash. Easiest route: a small shell function wrapping `fd` (find repo) + `fzf` (pick if ambiguous) + a `case` menu, with `zellij` (or `tmux`) handling the multiplex option.

The stack (all already-packaged tools)

Need	Tool	Why
Find repo by fuzzy name	`fd` + `fzf`	Fast, handles "my weird prompts" → `my-weird-prompts`
Action menu	bash `select` or `fzf`	`select` is built-in and gives exactly the numeric list you described
Open in Claude Code	`claude` CLI	Already installed
Open in file manager	`dolphin <path>`	KDE default
Open in VS Code	`code <path>`	Already installed
Open in terminal	`konsole --workdir <path>`	KDE default
Tiled multiplex	`zellij` **or** `tmux` **or** `konsole --split-view`	See below

Multiplex option — pick one

1. **Zellij** (recommended) — modern, declarative layouts via KDL. One layout file defines the three panes (Claude / Dolphin-in-terminal view / shell). Launch: `zellij --layout repo.kdl`. Easiest for "always same tiled presentation."
2. **tmux** — same idea, more ubiquitous, layout via `tmux new-session \; split-window \; ...`.
3. **Konsole split view** — native KDE, but scripting splits is clunkier than zellij/tmux. Skip unless you want zero new deps.

Note: a GUI file manager (Dolphin) can't live *inside* a terminal multiplexer pane. For a true tiled window with Dolphin + Claude + terminal side-by-side you need a **window-manager-level** tile, not a terminal multiplexer. Two realistic options:

- **KWin scripting / kstart + window rules**: launch Dolphin, Konsole-running-claude, and Konsole-shell, then use `kwin --replace` scripts or `wmctrl` / `kdotool` to place them. Fiddly on Wayland.
- **Compromise**: run everything inside one Konsole window using zellij/tmux panes (Claude + shell + `ranger` / `yazi` as the "file manager"). Much simpler, 100% reliable on Wayland. Recommended.

Minimal implementation

Drop this in `~/.config/fish/functions/openrepo.fish` or `~/.bashrc`:

```
openrepo() {
  local query="$*"
  local root="$HOME/repos"
  # Find candidate repo dirs (any .git parent) matching the query loosely
  local slug
  slug=$(echo "$query" | tr ' ' '-' | tr '[:upper:]' '[:lower:]')
  local match
```

```
  match=$(fd -t d -H '^\.git$' "$root" -x dirname {} \
          | fzf --filter="$slug" | head -n1)
  if [[ -z "$match" ]]; then
    # fallback: interactive fuzzy pick
    match=$(fd -t d -H '^\.git$' "$root" -x dirname {} | fzf)
  fi
  [[ -z "$match" ]] && { echo "No repo found for: $query"; return 1; }

  echo "Repo: $match"
  PS3=$'\nChoose action: '
  select action in \
      "Open in Claude Code" \
      "Open in File Manager" \
      "Open in VS Code" \
      "Open at Terminal" \
      "Multiplex (Claude + Files + Terminal)" \
      "Cancel"; do
    case $REPLY in
      1) (cd "$match" && claude) ; break ;;
      2) dolphin "$match" & disown ; break ;;
      3) code "$match" & disown ; break ;;
      4) konsole --workdir "$match" & disown ; break ;;
      5) konsole --workdir "$match" -e zellij --layout ~/.config/zellij/repo.kdl & disown ;
break ;;
      6) break ;;
    esac
  done
}
```

Zellij layout `~/.config/zellij/repo.kdl`:

```
layout {
  pane split_direction="vertical" {
    pane command="claude"
    pane split_direction="horizontal" {
      pane command="yazi"   // terminal file manager
      pane                  // plain shell
    }
  }
}
```

Usage:

```
$ openrepo my weird prompts
Repo: /home/daniel/repos/github/project/my-weird-prompts
1) Open in Claude Code
2) Open in File Manager
3) Open in VS Code
4) Open at Terminal
5) Multiplex (Claude + Files + Terminal)
6) Cancel
Choose action: 1
```

That's exactly the flow described.

"From a chatbox" variant

If you also want to invoke this from a non-terminal UI (e.g. KRunner, rofi, or an AI chat), wrap the same logic in a script and:

- **KRunner**: create a `.desktop` file with `Exec=konsole -e openrepo %u` — then Alt+Space → type "openrepo my weird prompts".
- **Rofi**: `rofi -dmenu -p "openrepo"` piped into the function.
- **From Claude Code / an MCP chat**: expose as a tiny local MCP tool that shells out to the same function. Overkill unless you're already in that surface.

Recommendation

1. Install `fd`, `fzf`, `zellij`, `yazi` (`apt install fd-find fzf; cargo install zellij yazi-fm` or use the `.deb`s).
2. Paste the `openrepo` function above.
3. Accept the compromise that "file manager" inside the multiplex is a TUI (`yazi`), not Dolphin — this is the only way to get a reliable tiled presentation on Wayland without fighting KWin.
4. Keep option 2 ("Open in File Manager") as the escape hatch that launches the real Dolphin window.

Total effort: ~15 minutes. No bespoke tool needed.

Sources

- `fzf` README — github.com/junegunn/fzf
- `zellij` layouts docs — zellij.dev/documentation/layouts.html
- `yazi` (terminal file manager) — yazi-rs.github.io
- bash `select` builtin — GNU Bash Reference Manual, §3.2.5.2
- KDE Konsole CLI flags — `man konsole` (`--workdir`, `-e`)
- Wayland window-placement limitations — KDE community wiki notes on KWin scripting under Wayland